SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1712
NAME: Preethi R REG NO: 192511352

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration: 1 Hour
Total Marks: 50

Annexure A

Scenario Based Assignment

Student 25: Preethi R | Register Number: 192511352

1. Scenario

A mobile gaming company is developing an AI opponent for a chess app aimed at casual players. The AI must compete against human players, make decisions within strict time limits (a few seconds per move), evaluate complex board positions with many possible moves, and the decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem.
ii. Analyze the computational challenges of large game trees.
iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.
iv. Design an evaluation function and justify the factors considered.
v. Recommend a strategy for balancing optimality and real-time performance.

2. Scenario

A disaster-response agency deploys AI-guided drones to airlift medical supplies to villages cut off by a wildfire. The environment is highly dynamic - access roads and forest trails may become impassable without warning, and smoke conditions affect travel cost. The drone has access to: a heuristic estimate (straight-line distance to each village), partial real-time updates about newly blocked trails, and variable movement costs depending on terrain and visibility. Due to the urgency of the situation, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic.

iii. Analyze the impact of heuristic accuracy on both algorithms.

iv. Discuss how dynamic changes (newly blocked trails) affect search performance.

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

3. Scenario

A large agricultural enterprise implements an AI system to optimize water distribution through a network of smart irrigation valves across thousands of acres. The objective is to minimize water wastage, energy consumption, and crop stress. The system must continuously adjust valve settings based on soil-moisture sensor readings. However, the search space is extremely large, weather patterns change dynamically, and the system may get stuck in locally optimal valve configurations.

Tasks:

Formulate this as an optimization problem and explain why traditional search is inefficient.

i. Explain how Hill Climbing would work in this scenario and identify its limitations.

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

iv. Recommend the best approach and justify your choice based on scalability and adaptability.

Code of Conduct:

I Preethi R(192511132)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

97

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

$20+20+25+18+14$

$= 97$

Name: Preethi R

Regno: 192511352

Course Code: CSA1712

CO-2 ASSESSMENT TOOL - 2

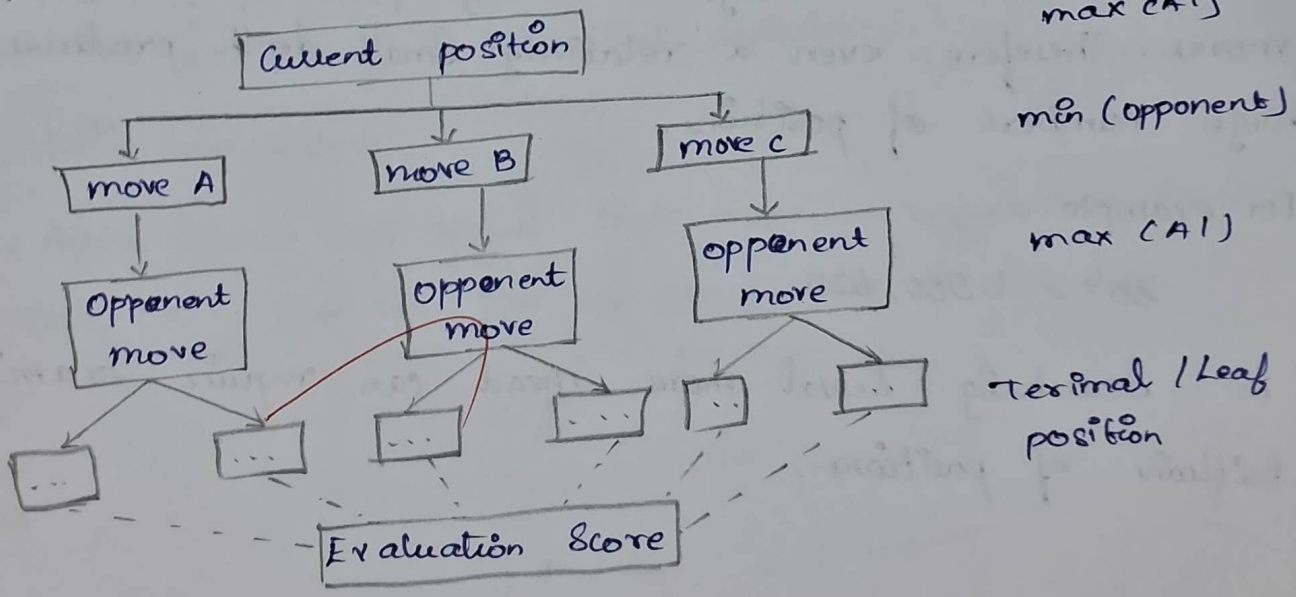
SCENARIO BASED ASSIGNMENT

1. AI Chess Opponent - Minimax and Alpha-Beta Pruning

i) The chess game can be represented as a game tree, where:

- Root node → Current chess board position.
- Nodes → possible board position after moves.
- Edges → Legal chess moves.
- Max player → AI opponent, trying to maximize its chance of winning.
- MIN player → Human player, assumed to make the best move to reduce the AI's advantage.
- Leaf nodes → positions reached after a certain depth of moves.
- Evaluation function → Gives a numerical score to each position.

Example:



Minimax assume that both players play optimally.
→ Max choose the move with the highest evaluation.
→ Min chooses the move with the lowest evaluation.

For a causal chess application the AI does not needed to search until checkmate. It can search to a limited depth, such as 4-8 plies, and use an evaluation function at the cutoff.

ii) Analyze the computational challenges of large game trees

Chess has a very large search space because a player can have many legal moves at each position.

If the:
→ Branching factor = b
→ Search depth = d

then the approximate number of nodes is: $O(b^d)$

For chess, the average branching factor is roughly 35 moves. Therefore, even a relatively small depth produces a huge numbers of positions.

For example:

$35^4 = 1,500,625$

So searching 5eval moves ahead can require examining millions of positions.

Main challenges:

1. Exponential growth - Number of positions increases rapidly with depth.
2. Limited time - Mobile games may allow only a few seconds per move.
3. Limited CPU / battery resource: Mobile device cannot perform unlimited computation.
4. Complex position - Materials alone cannot determine the quality of a chess position.
5. Time - depth trade off - Deeper search usually give better decision but takes longer.

Therefore, a plain minimax algorithm is often too expensive for real-time chess.

iii) Explain how Alpha- Beta pruning improve effeciency

Alpha- Beta pruning improves minimax by eliminating branches that cannot affect the final decision.

It maintains two values:

- Alpha (α) → Best value that max can guantee so far
- Beta (β) → Best value that min can guantee so far.

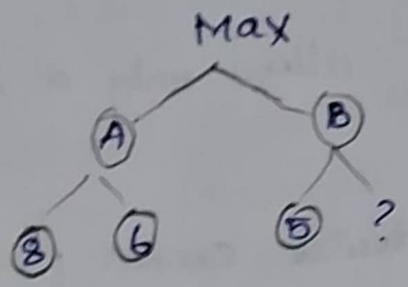
Initially:

$\alpha = -\infty$

$\beta = +\infty$

Example:

Suppose max is evaluating two moves:



For move A:

max (8, 6) = 8

So max already has a guaranteed value of 8.

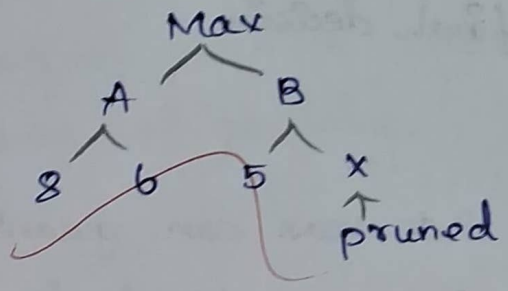
Now consider move B. MIN finds a value of 5.

Since MIN will choose the smaller value:

MIN (5, ?) ≤ 5

Therefore, Max will never select B because B can provide, almost 5, while A already give 8.

The remaining branch under B can be pruned.



The important condition is: $\alpha \geq \beta$

when this occurs, further exploration of that branch is unnecessary.

Efficiency:

Without pruning:

$O(b^d)$ with very good move ordering, Alpha-Beta can approach $O(b^{d/2})$

This means the AI can potentially search almost twice as deep within the same computational effort.

Design an evaluation function and justify the factors.

A chess evaluation function converts a board position into a numerical score.

A suitable evaluation function is:

$$Eval(S) = w_1 M + w_2 P + w_3 K + w_4 C + w_5 D + w_6$$

where

M = Materials advantages, P = Pawn structure, K = King safety, C = Center control, D = Development / mobility, S = Strategic positional advantages.

Import factors:-

Materials → Measure value of queens, rooks, bishops, knights and pawns.

King Safety → Prevents dangerous attacks and checkmate threats

Mobility → Rewards position with more useful legal moves

Center Control → control important central square

Pawn Structure → Identifies weak, isolated and passed pawns

Piece development → Encourage active placement of pieces.

A simple material model could be:

$$M = 9Q + 5R + 3B + 3N + 1P$$

where Q, R, B, N and P represent the material difference between the AI and opponent.

(3)

King Safety and tactical threats should receive high importance because losing the king or queen can dramatically change the position.

v) Recommend a strategy for balancing optimality and real time performance

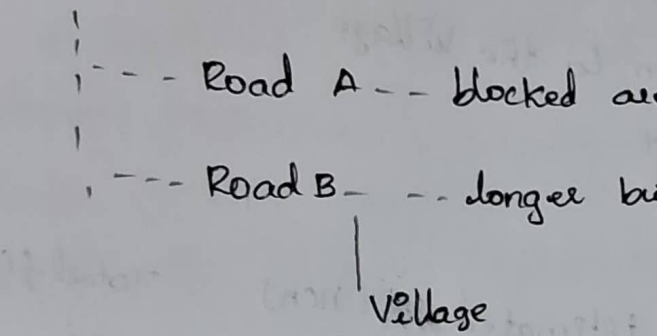
The best approach is depth-limited minimax + Alpha-Beta pruning + Good Move ordering + Iterative Deepening.

Recommended Strategy:

1. Use Alpha-Beta pruning to reduce unnecessary branches.
2. Use iterative deeping:
   → Search depth 2. then depth 3
   → Then depth 4. Continue until the time limit
3. order promising moves first.
4. Use a strong evaluation function at the search boundary.
5. Set a strict time limit.
6. If time expires, return the best move from the last completed search.
7. For a casual mobile game, prioritize fast and reasonable moves rather than perfect play.

Disaster – Response Drone Route Planning

Greedy Best-first Search selects the node that appears closest to the goal according to the heuristic where in this Scenario, the drone uses Straight line distance as its heuristic:



Greedy Search may choose road A because it appears geographically closer.

Strengths
→ Fast decision-making
→ Simple to implement
→ Requires relatively little computation
→ Useful when the destination must be reached quickly
→ Heuristic directly guides the drone toward the village.

Weaknesses
Greedy Search ignores the actual cost already spent.
It only consider:
→ Dangerous routes
→ High Cost terrain
→ Smoke-heavy area
→ Routes that become blocked
→ A geographically short route that takes longer in reality.

It is not guaranteed to find an optimal route.

ii) A* Combines:

1. Cost already spent
2. Estimated remaining cost

Its evaluation function is:

→ actual cost from drone's current location to node.
→ Estimated cost from to the village
→ Estimated total cost

For example:

Route	Actual cost g(n)	Estimated cost h(n)	Total f(n)
A	10	5	15
B	6	8	14

Thus A* Consider both distance and actual travel condition

iii) Impact of heuristic accuracy:

Greedy Best First Search

If the heuristic is accurate:

→ Search becomes faster
→ The drone moves towards promising route

If it is Inaccurate:

→ It can Seled poor route
→ It may repeatedly explore misleading directions.
→ It can choose dangerous or blocked paths.

is generally more robust because it considers: Even if the heuristic is impert, the actual cost provides additional information.

If the heuristic is:

* Admissible → It does not overestimate the actual remaining cost.

* Consistent → It maintain reliable cost relationship between nodes.

Under Suitable Condition, A* Can guantee an optimal Solution in a Static environment.

iv) Impact of dynamic changes

The environment is not Static

Forexample:

Initial:

Drone → Trail A → Village

After wild fire:

Drone → Trail A → Blocked

A route that was previously valid may become impossible.

Therefore, both algorithm may need to:

1. update the map
2. Remove blocked paths
3. Recalculate affected cost
4. Re-plan the route.

Problem with repeated Search

Running A* from the beginning every time a trail changes can be computationally expensive.

For a highly dynamic environment, approaches such as:

→ Incremental A*

→ D lite *

→ Dynamic replanning

can reuse previous search information and respond more quickly.

3. Smart - Irrigation optimization

optimization problem:

The Smart irrigation System can be formulated as an optimization problem. The main objective is to determine the best settings for thousands of irrigation valves while minimizing:

→ water wastage

→ Energy Consumption

→ Crop Stress.

The System uses Soil - moisture Sensors and weather information to continuously adjust the valves settings. Since every valve can have different Settings, the number of possible Combination becomes extremely large. Therefore, traditional exhaustive Search is computationally expensive and unsuitable for real - time Operation

i) Hill Climbing

Starts with one values configuration and repeatedly selects a neighboring configuration that improves the objective.

ii) Local maxima, plateaus & Ridges:

→ Local maximum: Get Stuck at a solution better than nearby solution but not the best overall.

→ Plateau: Many solution have nearly the same value, so no clear improvement exits.

→ Ridge: Improvement requires changing multiple values together.

iii) Simulated Annealing / Genetic Algorithm:

Simulated Annealing can accept worse solution temporarily to escape local optima. Genetic algorithm use Selection, crossover and mutation to explore many solution.

iv) Recommendation:

Genetic Algorithm is suitable because it can handle a large search space, avoid local optima, and adapt to changing weather and sensor condition.